

API key vs JWT (JSON Web Token)

API Key

- Es una clave única generada por el servidor, generalmente una cadena alfanumérica, que identifica a un **cliente(aplicación)** que quiere usar una API.
- **Uso:**
 - Se envía junto con cada solicitud como parte del **encabezado**, la **URL** o el **cuerpo del mensaje**.
 - Su propósito principal es autenticar al cliente (por ejemplo, a una aplicación JavaScript que consume la API), pero **no identifica un usuario específico**.
 - La *API key* se almacena en el servidor.
- **Ventajas:**
 - Fácil de implementar en PHP: basta con verificar que la clave recibida coincida con la almacenada en el servidor.
 - Ideal para casos simples como acceso público o control básico de consumo de la API:
 - Ej.: limitar las llamadas a la API desde un cliente

JWT (JSON Web Token)

- **JWT:** Es un estándar (RFC 7519) que permite transmitir información entre dos partes de forma segura, usando un token que está firmado digitalmente. Puede incluir información adicional, **como el ID del usuario, roles o permisos...**
- **Uso:**
 - No solo autentica al cliente(aplicación), sino también al **usuario** que realiza la solicitud.
 - Es útil para sistemas con usuarios autenticados.
 - El token se envía con cada solicitud, en el encabezado:

Authorization: Bearer <token>

- **Ventajas:**
 - Más seguro: como está firmado, no puede ser alterado sin invalidarlo.
 - Incluye información adicional (payload), como el nombre del usuario o permisos.
 - Tiene una fecha de expiración.
 - El **JWT** no necesita almacenarse en el servidor porque es **autocontenido** y lleva toda la información necesaria para la autenticación o autorización dentro del propio token.

Envío de **API key** en JavaScript

```
fetch("http://servidor.com/api ", {  
  method: "POST",  
  headers: {  
    "API_KEY": "clave_secreta_guardada"  
  }  
});
```

En el **servidor** se encontraría en:

```
// Captura el encabezado desde $_SERVER  
$recibidaAPIkey = $_SERVER['HTTP_API_KEY'];
```

Fichero **.htaccess** para garantizar que el servidor no filtra las cabeceras e impide su envío

```
<IfModule mod_setenvif.c>  
# Captura el valor del encabezado API_KEY y lo asigna a la variable HTTP_API_KEY  
  SetEnvIf API_KEY (.*?) HTTP_API_KEY=$1  
# Captura el valor del encabezado Authorization y lo asigna a la variable HTTP_AUTHORIZATION  
  SetEnvIf Authorization "(.*)" HTTP_AUTHORIZATION=$1  
</IfModule>
```

Consumir una API desde PHP

Si solo necesito realizar un GET puedo utilizar lo siguiente:

```
// URL de la API
$api_url = "http://localhost/api/v1/alimentos";

// Hacer la solicitud a la API y obtener la respuesta
$respuestaJSON = file_get_contents($api_url);

//Convertir JSON en array asociativo php
$arrayRespuesta = json_decode($respuestaJSON, true);
```

Para utilizar los verbos POST, PUT o DELETE desde PHP

- Existe extensión **cURL** de PHP.
- Ofrece muchos recursos para facilitar el trabajo.
- Ver los ejemplos en carpeta curl

Biblioteca externa que se puede instalar con Composer

- Biblioteca externa en PHP, **Guzzle**, que proporciona una interfaz para realizar solicitudes HTTP con diferentes métodos y opciones.

cURL

cURL es una extensión que permite realizar peticiones HTTP a otros servidores desde tu aplicación. Proporciona una manera flexible y eficiente de interactuar con APIs o descargar datos de recursos remotos.

Características de cURL:

1. **Soporta múltiples protocolos:** HTTP, HTTPS o FTP entre otros.
2. **Configurable:** Permite personalizar aspectos de la solicitud como encabezados, métodos (GET, POST, PUT, DELETE), autenticación, y manejo de cookies.
3. Útil para consumir APIs RESTful.

```
// Inicializa cURL
$ch = curl_init();

// Configura la URL y otras opciones
curl_setopt($ch, CURLOPT_URL, "http://localhost/api/recurso");
// Devuelve la respuesta en lugar de imprimirla
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

// Ejecuta la solicitud
$response = curl_exec($ch);

// Maneja errores, si los hay
if (curl_errno($ch)) {
    echo 'Error: ' . curl_error($ch);
} else {
    echo 'Respuesta: ' . $response;
}

// Cierra la conexión
curl_close($ch);
```

1. Funciones principales de cURL para trabajar con APIs

1. `curl_init()`

- Inicializa una nueva sesión de cURL.
- Devuelve un identificador de recurso para configurarlo posteriormente.

```
$ch = curl_init();
```

2. `curl_setopt()`

- Configura opciones para la solicitud cURL.
- Recibe tres parámetros:
 1. **Recurso de cURL** (`$ch`).
 2. **Constante de opción** (`CURLOPT_*`).
 3. **Valor de la opción**.

```
curl_setopt($ch, CURLOPT_URL, "https://api.ejemplo.com/recurso");  
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
```

3. `curl_exec()`

- Ejecuta la solicitud configurada.
- Devuelve la respuesta del servidor o `false` en caso de error.

```
$response = curl_exec($ch);
```

4. `curl_errno()` y `curl_error()`

- Devuelven el número de error o el mensaje de error de la última operación.

```
if (curl_errno($ch)) {  
    echo "Error: " . curl_error($ch);  
}
```

5. `curl_close()`

- Cierra la sesión de cURL y libera recursos.

```
curl_close($ch);
```

2. Opciones de configuración (CURLOPT_) comunes para APIs

1. CURLOPT_URL:

- Establece la URL a la que se enviará la solicitud.

```
curl_setopt($ch, CURLOPT_URL, "https://api.example.com/recurso");
```

2. CURLOPT_RETURNTRANSFER:

- Si se establece en `true`, la respuesta se devuelve como una cadena en lugar de imprimirse.

```
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
```

3. CURLOPT_HTTPHEADER:

- Especifica encabezados personalizados para la solicitud.

```
$headers = [  
    "Authorization: Bearer tu_token",  
    "Content-Type: application/json"  
];  
curl_setopt($ch, CURLOPT_HTTPHEADER, $headers);
```

4. CURLOPT_POST y CURLOPT_POSTFIELDS:

- Envía datos en una solicitud POST.

```
curl_setopt($ch, CURLOPT_POST, true);  
curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode(["key" =>  
    "value"]));
```

5. CURLOPT_CUSTOMREQUEST:

- Define métodos HTTP personalizados como PUT, DELETE, etc.

```
curl_setopt($ch, CURLOPT_CUSTOMREQUEST, "DELETE");
```

HATEOAS

HATEOAS es un acrónimo que significa **Hypermedia as the Engine of Application State** (Hypermedia como el Motor del Estado de la Aplicación).

El diseño de **APIs RESTful** que mejora el uso de la API para el cliente al incluir enlaces dinámicos en las respuestas de la API.

Estos enlaces guían al cliente sobre las posibles acciones que puede realizar a continuación, basándose en el estado actual de los recursos.

¿Qué es y cómo funciona?

HATEOAS define que una API RESTful debe proporcionar suficiente información dentro de sus respuestas para que el cliente pueda descubrir y navegar por los recursos disponibles sin necesidad de conocer de antemano las rutas o la lógica del servidor. Esto se logra mediante **la inclusión de hipermedia (enlaces)** en las respuestas.

Imagina que haces una solicitud a un API de gestión de usuarios:

Solicitud: GET /usuarios/1
Respuesta con HATEOAS:

```
{
  "id": 1,
  "nombre": "Juan Pérez",
  "email": "juan.perez@example.com",
  "links": [
    { "rel": "self", "href": "/usuarios/1" },
    { "rel": "editar", "href": "/usuarios/1/editar" },
    { "rel": "eliminar", "href": "/usuarios/1/eliminar" },
    { "rel": "amigos", "href": "/usuarios/1/amigos" }
  ]
}
```

En esta respuesta:

- El cliente sabe cómo obtener la información completa del usuario (self).
- También puede editar o eliminar al usuario.
- Incluso puede explorar los amigos de este usuario.

El cliente no necesita codificar manualmente las URLs ni conocer de antemano la estructura completa de la API. Puede navegar entre recursos y realizar acciones siguiendo los enlaces proporcionados.